TDT4265 Mini-Project

Snow Pole Detection for Autonomous Driving

Gustavo Garzo Chumilla Jordan Ikukele Bomolo Mia

NTNU – Computer Vision and Deep Learning

April 2026

Outline

Presentation roadmap

1. Background & Motivation
2. Approach / Strategy
3. Data Analysis
4. Methods & Models
5. Results
6. Discussion
7. Key Learning Points
8. Sustainability

Background & Motivation

- Autonomous driving systems perform well in ideal road conditions, but snowy roads remain challenging because boundaries and lane markings become hard to see.
- In winter, **snow poles** provide a robust visual cue for road localisation because they remain visible even when the road surface is covered.
- Our project therefore focuses on **single-class object detection** of snow poles in realistic winter driving scenes from Trondheim.

Main challenge: poles are thin, visually small, and often far from the camera.

The final goal is not only accuracy, but also a detector that could realistically support **real-time edge deployment**.

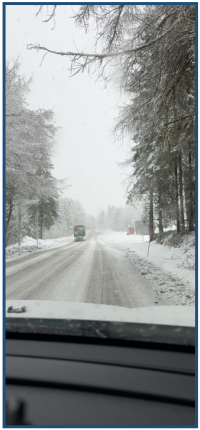
Problem framing

- task: one-class object detection
- application: winter road localisation
- constraint: edge-ready, real-time inference
- evaluation: Precision, Recall, mAP@50 and mAP@0.5:0.95

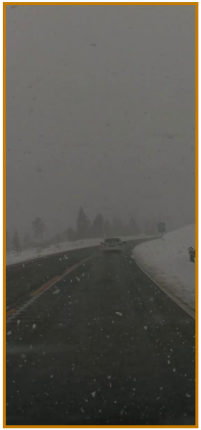
Background & Motivation – Visual Examples

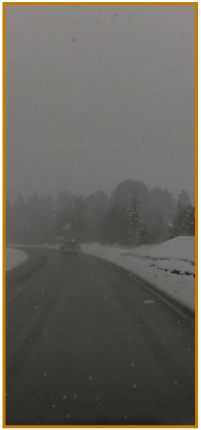
Representative training images

Road_poles_iPhone



roadpoles_v1





Approach / Strategy

Project strategy

- We first treated the task as a **dataset understanding problem**, not only a model training problem.
- We analysed **both available leaderboards separately**: `Road_poles_iPhone` and `roadpoles_v1`.
- The goal of the EDA was to understand whether both datasets had similar object scale, geometry and scene structure.

Why this matters

The course brief explicitly asks for model development to be guided by exploratory data analysis, so this step is part of the methodology rather than a side note.

Pipeline

Problem framing

↓

EDA on both datasets

↓

Resolution and model selection

↓

Training, validation and leaderboard submission

Approach / Strategy – Key Reasoning

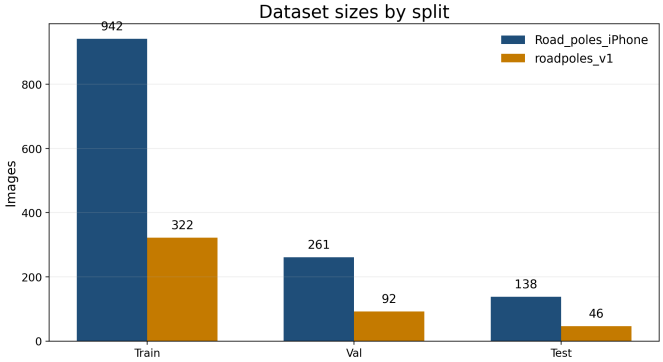
What the EDA needed to answer

- Are the two datasets similar enough that we should interpret them as the same task?
- Is the main difficulty object density, object scale, object geometry, or scene structure?
- Which data properties are most likely to explain differences in final leaderboard performance?

How the analysis informs the rest of the presentation

- If poles are very small, input resolution becomes a critical design choice.
- If one dataset is geometrically harder, lower results there do not necessarily mean the model is worse overall.
- This gives context for the model and training decisions shown in the next section.

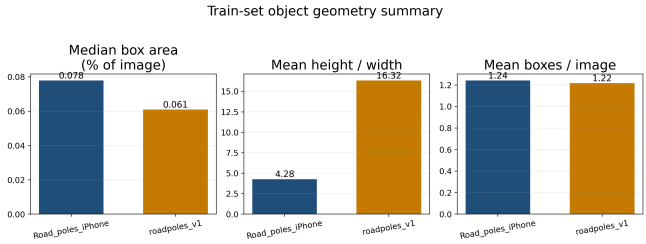
Data Analysis – Dataset Overview



Key observations

- `Road_poles_iPhone` is the larger dataset:
  - 942 train
  - 261 val
  - 138 test
- `roadpoles_v1` is smaller:
  - 322 train
  - 92 val
  - 46 test
- The test labels are hidden in both datasets, so the EDA must rely on train and validation splits only.

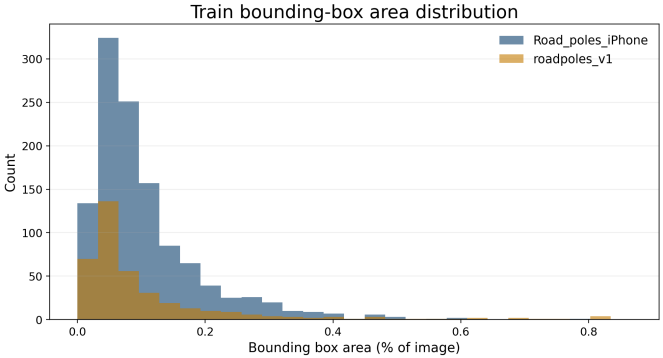
Data Analysis – Object Geometry



Why this is relevant

- `roadpoles_v1` is the **harder benchmark geometrically**:
  - smaller median box area
  - much larger height/width ratio
- Both datasets are sparse at around **1.2 poles per image**.
- The difficulty is therefore not crowded scenes, but detecting small and thin roadside objects.

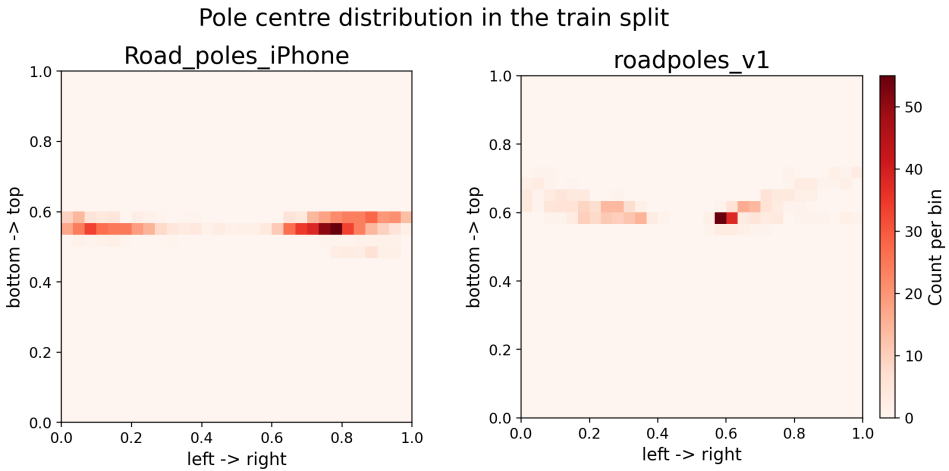
Data Analysis – Object Scale



Interpretation

- In both datasets, poles occupy only a **tiny percentage of the image area**.
- This confirms that the task is dominated by **small-object detection**.
- It also helps explain why image resolution is an important variable in the methods section.

Data Analysis – Spatial Distribution



Pole centres cluster in a narrow horizontal band in both datasets, consistent with road-side object placement.

Data Analysis – Spatial Distribution Insights

Interpretation

- The two datasets are not random image collections, but structured driving scenes with a clear spatial bias.
- This matters when interpreting results: errors are more likely to come from **small-object visibility and localisation** than from heavy clutter.
- These observations directly motivate the model and training choices shown in the next section.

Overview – Two Architectures

Model 1 – YOLOv8s

- Single-stage detector
- You Only Look Once – one forward pass
- Designed for real-time edge deployment
- **Dataset:** Road poles iPhone (942 train / 261 val)

Model 2 – Faster R-CNN

- Two-stage detector
- Region Proposal Network + classification
- ResNet-50 + FPN backbone
- **Dataset:** roadpoles v1 (322 train / 92 val)

Both models fine-tuned from ImageNet/COCO pretrained weights

Model 1 – YOLOv8s Architecture

Why YOLOv8s?

- Poles occupy ≈2% of image width – need enough model capacity
- YOLOv8**n** (nano) too weak for small objects
- YOLOv8**s** (small, 11.2M params) – best speed/accuracy tradeoff
- Must run on edge device in real-time

Key training decisions:

- `imgsz=1280` – more pixels per pole
- 150 epochs with early stopping (patience=30)
- Mosaic, scale, colour jitter augmentations

Model stats

- Parameters: 11.2M
- Input size: 1280×1280
- Classes: 1 (pole)
- Pretrained: COCO
- Batch size: 8
- Optimiser: auto (AdamW)

Model 1 – YOLOv8s Training Config

Training code

```
model = YOLO("yolov8s.pt")
model.train(
    data="data.yaml",
    epochs=150,
    imgsz=1280,
    batch=8,
    patience=30,
    hsv_h=0.015, hsv_s=0.7, hsv_v=0.4,
    fliplr=0.5, mosaic=1.0, scale=0.5,
    translate=0.1,
    device=0
)
```

- Batch size reduced to 8 due to `imgsz=1280` VRAM requirements
- Fine-tuned from COCO pretrained weights
- Trained on NVIDIA RTX 4090 (Cybele lab)

Model 2 – Faster R-CNN Architecture

Two-stage detection pipeline:

1. **Stage 1 – RPN:** Scans feature map, proposes candidate regions containing objects
2. **Stage 2 – ROI Head:** Classifies each proposal and refines the bounding box

FPN backbone: Extracts features at multiple scales simultaneously – important for poles at different distances **Key difference from YOLO:**

Two passes = slower but theoretically more accurate on small objects

Architecture

- Backbone: ResNet-50 + FPN
- Pretrained: COCO
- Head: replaced for 2 classes (background + pole)
- Optimiser: SGD, lr=0.005
- Scheduler: StepLR
- Epochs: 100

Model 2 – Faster R-CNN Training Config

Custom dataset class – YOLO labels to Faster R-CNN format

```
# Convert normalised YOLO format to absolute pixel coords
x1 = (cx - bw/2) * W
y1 = (cy - bh/2) * H
x2 = (cx + bw/2) * W
y2 = (cy + bh/2) * H
# Labels: 0 = background, 1 = pole
```

Augmentations (train only)

```
train_transform = T.Compose([
    T.ToTensor(),
    T.ColorJitter(brightness=0.4, contrast=0.4,
                  saturation=0.3, hue=0.1),
    T.RandomHorizontalFlip(p=0.5),
])
```

Results – Validation Set Metrics

Model	Precision	Recall	mAP@50	mAP@0.5:0.95
YOLOv8s	0.957	0.976	**0.990**	**0.788**
Faster R-CNN	0.626	0.679	0.993	0.626

Table: Validation set results on respective datasets

- Both models achieve **mAP@50** $>$ **0.99** – poles are reliably detected
- YOLO significantly better on **mAP@0.5:0.95** (0.788 vs 0.626)
- Faster R-CNN higher Precision but lower Recall

Results – Leaderboard

Road poles iPhone leaderboard

- Submitted YOLOv8s predictions
- Score: $\approx$**69%**

roadpoles v1 leaderboard

- Submitted Faster R-CNN predictions
- Score: $\approx$**66%**

Note: Test set scores lower than validation – expected due to unseen weather/lighting conditions in test set

Results – mAP@50 vs mAP@0.5:0.95 Gap

The gap between the two metrics tells us how **precisely the boxes fit** the poles:

- **mAP@50:** Box needs to overlap ground truth by $\geq$50% – both models pass easily
- **mAP@0.5:0.95:** Averaged over IoU thresholds 0.50 to 0.95 – requires tight localisation

YOLO produces **tighter bounding boxes** – likely due to larger training set and higher image resolution

	@50	**@0.5:0.95**
YOLOv8s	0.990	0.788
Faster R-CNN	0.993	0.626

Gap for YOLO: 0.202
Gap for RCNN: 0.367

Discussion – Why YOLO Outperformed Faster R-CNN

Three key factors

1. **Dataset size:** YOLO trained on 942 images vs 322 for Faster R-CNN. More data = better generalisation, especially for small objects.
2. **Image resolution:** YOLO used `imgsz=1280` vs default 800 for Faster R-CNN. More pixels per pole directly improves detection.
3. **Augmentation:** YOLO used mosaic augmentation which creates composite training images – effectively increases dataset diversity.

Faster R-CNN is theoretically stronger on small objects due to FPN + two-stage design, but needs more data and tuning to demonstrate this advantage.

Discussion – Limitations & Future Work

Current limitations:

- Faster R-CNN trained on smaller dataset – unfair comparison
- Test set gap suggests domain shift (weather, lighting)
- Faster R-CNN too slow for real-time edge deployment as-is
- No pseudo-labelling of unlabelled RoadPoles-MSJ data

Future improvements:

- Increase Faster R-CNN `imgsz` to 1024/1280
- Pseudo-label RoadPoles-MSJ with best YOLO model
- Try MobileNet backbone for faster inference
- Test time augmentation (TTA) for better test scores
- Ensemble YOLO + Faster R-CNN predictions

Key Learning Points

1 – Resolution matters more than model size

Increasing `imgsz` from 640 to 1280 had a larger impact than switching from YOLOv8n to YOLOv8s. For tiny objects like snow poles, giving the model more pixels to work with is the single most effective improvement.

2 – Understanding two-stage detection

Implementing Faster R-CNN manually revealed how the RPN and ROI pooling work together – a much deeper understanding than using YOLO through a high-level API. The label format conversion (YOLO normalised to absolute pixel coordinates) highlighted how different frameworks expect data.

3 – Applied deep learning is mostly engineering

A significant portion of the project was path issues, yaml configuration, CUDA driver mismatches, and dataset format debugging. These are the real-world challenges in any CV deployment scenario.

Sustainability – Compute Energy

Hardware used:

- NVIDIA RTX 4090 (Cybele lab)
- TDP: 450W

Approximate training time:

- YOLOv8s (150 epochs, imgsz=1280): ≈2.5h
- Faster R-CNN (100 epochs): ≈1.5h
- Total: ≈**4 hours**

Energy consumed:

$$E = 0.45\,\text{kW} \times 4\,\text{h} = \mathbf{1.8}\,\text{kWh}$$

How long could we code?

MacBook Pro M3 consumption:
≈**45 W under load**

$$t = \frac{1800\,\text{Wh}}{45\,\text{W}} = \mathbf{40}\,\text{hours}$$

≈ a full work week of laptop use

Relatively low footprint – fine-tuning from pretrained weights avoided training from scratch

Summary

Model	mAP@50	mAP@0.5:0.95	Speed	Edge-ready
YOLOv8s	0.990	0.788	Fast	**Yes**
Faster R-CNN	0.993	0.626	Slow	**No**

- **Best model for deployment:** YOLOv8s – higher mAP@0.5:0.95, real-time capable
- **Biggest improvement lever:** image resolution for small object detection
- **Energy cost:** 1.8 kWh total $\approx$ 12 km in a Tesla Model Y

Thank you for listening